Laboratório Prolog · Aula 13

Matheus Torrezan Izepe · RA 241704362 · Turma ESOFT6S-C
Gerado em 20/10/2026, 19:40:09 · tempo registrado no cronômetro: 0:00

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(L, funcional) responde L = scheme. paradigma(java, logico) responde false, pois a base registra Java como orientado_a_objetos.

Saída do ola.pl

```
Saída esperada: linguagem(prolog), linguagem(scheme), linguagem(java);
paradigma(prolog,logico); paradigma(scheme,funcional); paradigma(java,logico) → false.
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) pergunta qual valor P está associado ao paradigma de Prolog; responde P = logico. paradigma(prolog, logico) pergunta se esse fato específico é verdadeiro; responde true.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Significa que o programa não possui um fato/regra que prove paradigma(java, logico). Nesta base, Java está associado a orientado_a_objetos. O mundo fechado do Prolog trata o que não pode ser provado como falso; isso não é uma afirmação universal sobre todas as classificações possíveis de Java.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) terá 7 respostas, na ordem: lucia, marta, tereza, joana, bia, ana, dora. A conjunção (mae(M,F), mulher(F)) terá 4 pares, na ordem dos fatos: (lucia,tereza), (marta,joana), (tereza,bia) e (joana,ana).

Saída do familia.pl

```
Saídas principais: mae(joana,jake) true; mae(jake,joana) false; pai(maria,jake) false;
mae(joana,_) → jake, ana; pai(_,jake) → jorge; mulher(_) → lucia, marta, tereza, joana,
bia, ana, dora. Conjunções: (mae(joana,X),homem(X)) false; (pai(vern,Y),mulher(Y)) → joana;
(mae(_,Filha),mulher(Filha)) → tereza, joana, bia, ana conforme a ordem dos fatos.
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

Pergunta quem é mãe de alguém que consta como mulher: Mae é a mãe e Filha é a filha, que também precisa satisfazer mulher(Filha). Há 3 respostas: (marta, joana), (tereza, bia) e (joana, ana).

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

É a hipótese do mundo fechado: se a consulta não pode ser provada com os fatos e regras disponíveis, o Prolog responde false. Em situações reais, uma base incompleta pode não conter um fato verdadeiro; tratá-lo como falso pode confundir desconhecido com falso.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

lucia, marta, tereza, joana, bia, ana, dora. A ordem acompanha a ordem em que os fatos mulher/1 aparecem no programa, pois o Prolog busca as cláusulas de cima para baixo.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X,jake): X = vern? Não: os progenitores de jake são joana e jorge; joana tem como irmão apenas paulo (mesmos pais? joana e paulo compartilham marta e vern), mas tio exige homem, então X = paulo. primo(leo,X): os pais de leo são dora e paulo; os irmãos de paulo são joana, logo os filhos de joana (jake e ana) são primos de leo. Também os irmãos de dora não estão na base. Respostas: tio(paulo,jake); primo(leo,jake) e primo(leo,ana).

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Falha: f(X,X)=f(a,b), pois X teria de ser a e b ao mesmo tempo; [A|B]=[] falha porque lista vazia não tem cabeça/cauda. Unificam: f(X,X)=f(a,Y), com X=a e Y=a; e 2+3=5 também unifica estruturalmente? Não: = não calcula, então o termo +(2,3) não é igual ao número 5; falha.

Saídas de regras.pl e unificacao.pl

```
Regras: tio(_,jake) → paulo; primo(leo,_) → jake, ana. avo(_,jake) → osvaldo, lucia,
jorge? (avós diretos: progenitores dos pais: osvaldo/lucia de vern? jake parents joana/jorge,
so avós: marta, vern, jorge has no parents in base; thus marta and vern). avo(osvaldo,_) →
vern, tereza, joana, paulo, jake, ana, leo? consulte a cadeia conforme fatos. Unificação:
f(X,X)=f(a,b) falha; f(X,X)=f(a,Y) unifica; [A|B]=[] falha; 2+3=5 falha por ser estrutural.
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Precisa de duas cláusulas para cobrir relações diretas e relações indiretas. Caso base: ancestral(X,Y) :- progenitor(X,Y). Caso recursivo: ancestral(X,Y) :- progenitor(X,Z), ancestral(Z,Y), que segue pela cadeia de progenitores.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, a regra também permite irmao(jake,jake), porque jake compartilha os mesmos pais consigo mesmo. X \== Y impede que uma pessoa seja considerada irmã/irmão de si mesma.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Na primeira, X teria de ser simultaneamente a e b, o que é impossível. Na segunda, X unifica com a e Y também fica a, porque o segundo argumento de f(a,Y) precisa ser igual ao mesmo X.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Não da mesma forma com uma função comum: em Java ou Scheme, normalmente definimos parâmetros de entrada e calculamos um retorno numa direção. A relação avo/2 pode ser consultada com o avô conhecido para encontrar netos, ou com o neto conhecido para encontrar avós, porque variáveis e unificação permitem diferentes argumentos desconhecidos (respeitadas as limitações da execução).

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

Quatro respostas, dividindo [1,2,3] em todos os pontos possíveis: X=[], Y=[1,2,3]; X=[1], Y=[2,3]; X=[1,2], Y=[3]; X=[1,2,3], Y=[].

Saída do listas.pl

```
membro(b,[a,b,c]) true; membro(z,[a,b,c]) false; membro(_, [a,b,c]) → a,b,c.
tamanho([a,b,c],3); tamanho([],0). concatena([1,2],[3,4],[1,2,3,4]); concatena(_, [3],
[1,2,3]) → X=[1,2]; concatena([1],_,[1,2,3]) → Y=[2,3]; concatena(_,_,[1,2,3]) tem 4
divisões. inverte([1,2,3],[3,2,1]).
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

Caso base: tamanho([],0). Passo recursivo: tamanho([_|Cauda],N) chama tamanho(Cauda,M) e define N is M+1. A cada chamada fica pendente somar 1 ao tamanho da cauda; ao chegar à lista vazia, N=0 e as chamadas retornam calculando os tamanhos.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Há quatro respostas porque a lista final pode ser dividida em X e Y em quatro posições, incluindo antes do primeiro e depois do último elemento. O anexar do Scheme é uma função que recebe argumentos e devolve um resultado; não costuma inverter a direção para descobrir todas as divisões de uma lista final. A relação Prolog pode trabalhar em diferentes direções via unificação e backtracking.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

Backtracking: o Prolog tenta primeiro a cláusula que casa X com a cabeça e depois, ao pedir outras respostas, volta e percorre recursivamente a cauda.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3 apenas unifica X com a expressão simbólica +(2,3), sem calcular. X is 2 + 3 calcula e responde X = 5. fatorial(N,120) dá erro de instanciação porque a regra usa comparações e aritmética com N ainda não conhecido.

Saída do aritmetica.pl

```
=: X = 2+3; is: X=5; 5 =:= 2+3 true; 2+3 = 5 false; 7 mod 3 → 1; 7 // 2 → 3; 2^10 → 1024;
fatorial(5,120); fatorial(0,1); fib(10,55); soma([1,2,3,4],10). fatorial(N,120) com N livre e
_ is _+1 geram erro de instanciação.
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

=: unifica termos sem fazer contas; por exemplo, X = 2+3 deixa X ligado à expressão 2+3. is: calcula o lado direito e unifica o resultado, por exemplo X is 2+3 dá X=5. =:= compara valores numéricos após avaliar ambos os lados; por exemplo 5 =:= 2+3 é verdadeiro.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

fatorial(N,120) tenta calcular com N desconhecido e chega a uma operação aritmética que exige N instanciado, gerando erro. concatena(X,[3],[1,2,3]) só compara e decompõe estruturas de listas por unificação, podendo descobrir X=[1,2] sem calcular números.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Ambos usam casos base e recursão para reduzir o problema até chegar ao caso base. Em Haskell, fact é uma função que recebe n e retorna o fatorial. Em Prolog, fatorial(N,F) é uma relação entre entrada e resultado, mas as operações aritméticas (>, is) exigem valores suficientemente conhecidos, então não é totalmente reversível.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2(X) falha com X livre porque tenta provar \+ casado(X) antes de gerar homens; como há alguém casado, a negação falha. solteiro1(X) produz candidatos homem primeiro e depois verifica que não são casados. maximo2(5,3,M) dá duas respostas (M=5 e M=3); maximo3(5,3,3) dá true por causa do corte após X>=Y.

Saída de controle.pl e mensagem de ancestral_ruim

```
solteiro1: osvaldo, paulo, jake (homens não listados como casados). solteiro2: false.
maximo1(5,3,M) → M=5; maximo2 → M=5 e M=3; maximo3 → M=5. maximo1(5,3,3) false;
maximo3(5,3,3) true pelo corte. ancestral_bom(osvaldo,jake) sim; ancestral_ruim pode estourar
a pilha.
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores existem 24 colorações válidas para o mapa descrito. Com 2 cores não existe nenhuma, pois o grafo contém triângulos de regiões mutuamente adjacentes, que exigem pelo menos 3 cores.

Saída do mapa.pl

```
Mapa: com 3 cores há 24 colorações; com 2 cores há 0; com 4 cores há 1728 (para as fronteiras
e a ordem de regiões descritas no código).
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog percorre amigos em ordem ana, bia, caio para X e Y. ana/ana falha em X \== Y; ana/bia passa por X\==Y, mas falha porque ana gosta de café e bia de chá. O backtracking tenta ana/caio, que compartilha café e gera a primeira resposta; depois segue para as combinações seguintes e encontra caio/ana, a segunda resposta. A meta gosta(Y,B) falha quando os gostos não coincidem e a busca volta às escolhas anteriores.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

solteiro1 gera primeiro cada homem e testa se ele não é casado. solteiro2 avalia a negação com X ainda livre; como existem fatos casado/2, a negação como falha não consegue concluir que nenhum casado(X) vale e falha. solteiro1 afirma que osvaldo é solteiro porque a base não contém casado(osvaldo), embora ele seja pai de vern: ser pai não implica estar casado na base.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O corte compromete a busca assim que X>=Y é verdadeiro, antes de verificar se o terceiro argumento corresponde a X. Em maximo3(5,3,3), a primeira cláusula passa no teste e executa !; depois X (5) não unifica com o terceiro argumento 3, mas a segunda cláusula já foi descartada. Corrija verificando também o resultado antes do corte: maximo3(X,Y,M) :- X>=Y, M=X, !. (ou usando M=X antes do corte, conforme a sintaxe pretendida).

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim chama ancestral_ruim recursivamente antes de reduzir o problema com progenitor/2; essa recursão à esquerda repete a mesma busca indefinidamente e esgota a pilha. ancestral_bom primeiro encontra um progenitor e avança para um ancestral mais próximo do destino, até atingir o caso base.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

Com 3 cores: 24 colorações; com 2 cores: 0; com 4 cores: 1728. É declarativo porque o código descreve as restrições de uma coloração válida, não uma sequência explícita de passos ou laços; o mecanismo de busca/backtracking do

Prolog encontra as combinações.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No paradigma lógico, concatena/3 descreve uma relação entre duas listas e a lista resultante, podendo inferir qualquer argumento que esteja desconhecido. No funcional, anexar do Scheme recebe argumentos e calcula/devolve um resultado numa direção definida.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

1) A aritmética não é totalmente relacional: fatorial(N,120) dá erro com N desconhecido por causa de is/comparações. 2) A ordem das metas e a recursão importam: solteiro2 falha com variável livre e ancestral_ruim pode entrar em recursão infinita e estourar a pilha.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas baseados em relações, regras, dedução, parentesco, restrições e busca de soluções, pois a base de fatos e regras é natural. Não escolheria para interfaces gráficas, aplicações de negócio convencionais ou processamento numérico intensivo, em que Java costuma oferecer ecossistema, controle de fluxo e desempenho mais adequados.

Exercícios opcionais

Opcionais: resultado dos testes

```
Os sete exercícios opcionais estão implementados nos campos E1–E7 abaixo. Ao colar todos os
predicados no arquivo enunciado e executar, os testes devem indicar ok.
```

E1 Regras de família: código escrito

```
irma(X, Y) :- mulher(X), irmao(X, Y).
tia(X, Y) :- mulher(X), irmao(X, P), progenitor(P, Y).
```

E2 Listas: código escrito

```
ultimo([X], X).
ultimo([_|T], X) :- ultimo(T, X).
```

E3 As funções da Aula 12: código escrito

```
soma_quadrados([], 0).
soma_quadrados([H|T], S) :- soma_quadrados(T, S1), S is S1 + H*H.

pares([], []).
pares([H|T], [H|P]) :- H mod 2 =:= 0, pares(T, P).
pares([H|T], P) :- H mod 2 =\= 0, pares(T, P).
```

E4 Aritmética: código escrito

```
mdc(A, 0, A) :- A ⩾ 0.
mdc(A, B, D) :- B > 0, R is A mod B, mdc(B, R, D).

potencia(_, 0, 1).
potencia(B, E, R) :- E > 0, E1 is E - 1, potencia(B, E1, R1), R is B * R1.
```

E5 Grafo com ciclos: código escrito

```
caminho(A, B) :- caminho_visitados(A, B, [A]).
caminho_visitados(A, B, _) :- liga(A, B).
caminho_visitados(A, B, Visitados) :-
    liga(A, C),
    \+ membro(C, Visitados),
    caminho_visitados(C, B, [C|Visitados]).
```

E6 Mapa menor: código escrito

```
mapa_sul(Paleta, [RS, SC, PR]) :-
    membro(RS, Paleta), membro(SC, Paleta), membro(PR, Paleta),
    RS \== SC, SC \== PR.
```

E7 Prolog como banco de dados: código escrito

```
colega(X, Y) :- funcionario(X, D, _), funcionario(Y, D, _), X \== Y.
ganha_mais(X, Y) :- funcionario(X, _, SX), funcionario(Y, _, SY), SX > SY.
total_depto(D, Total) :-
    findall(S, funcionario(_, D, S), Salarios),
    soma(Salarios, Total).
```